

Laboratório 2

Ponteiros

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz
marcosleipnitz@ic.ufrj.br



UFRJ

UNIVERSIDADE FEDERAL
DO RIO DE JANEIRO



INSTITUTO DE
COMPUTAÇÃO
UFRJ



Exercício

Escreva um programa em C que:

1. Leia um número **inteiro positivo** digitado pelo usuário.
2. Mostre na tela o valor desse número em formato **hexadecimal**.
3. Imprima na tela o **conteúdo da memória** desse número, **byte a byte**, na ordem em que eles estão armazenados, mostrando o **endereço** e o **valor em hexadecimal** de cada byte.
4. Determine e informe se o sistema é *little-endian* ou *big-endian*.
(<https://en.wikipedia.org/wiki/Endianness>)

Obs.: Utilize ponteiros para acessar os bytes da variável.

Em C, o compilador pode inserir **bytes de preenchimento** (padding) entre os campos de uma estrutura (**struct**) e também ao final dela, para atender a requisitos de alinhamento de memória. Isso pode fazer com que o tamanho total de uma struct (**sizeof**) seja maior que a soma dos tamanhos de seus campos. Mais informações sobre alinhamento de memória:

https://en.wikipedia.org/wiki/Data_structure_alignment

Implemente um programa em C que:

1. Defina um tipo **struct** chamado **Packet** com quatro campos de tipos diferentes, nesta ordem: char, int, short e double.
2. Crie uma variável desse tipo com valores quaisquer.

- 3. Calcule e imprima** na tela as seguintes informações, usando apenas aritmética de ponteiros e a função `sizeof`:
- a. O **offset de cada campo** (quantos bytes a partir do início da estrutura), obtido pela diferença de endereços.
 - b. O **tamanho de cada campo**.
 - c. A quantidade de **padding entre os campos**.
 - d. O **padding de cauda** (quantidade de bytes entre o final do último campo e o final da estrutura).
 - e. O **total de padding da estrutura** (tamanho total menos a soma dos tamanhos dos campos).